Final Project Report

FitBuddy : AI-Powered Personalized 7-Day Fitness Plan Generator

Group ID: GACSE-114

Subject: AI-Powered Personalized Fitness Systems

Date: March 2, 2026

Table of Contents:

S No.	Content	Page No.
1	Executive Summary	2
2	Problem Statement	2
3	Objectives	3
4	Scope of the Project	3
5	Technology Stack	4
6	System Architecture	5
7	Module Description	8
8	Database Design	9
9	User Interface Description	10
10	API Specification	11
11	Validation and Error Handling	12
12	Security Considerations	12
13	Testing Strategy	13
14	Key Features Implemented	13
15	Limitations	14
16	Future Enhancements	14
17	Conclusion	15
18	References	15
19	Appendix	15

1. Executive Summary :

FitBuddy is an AI-powered web application designed to generate personalized 7-day fitness plans tailored to individual user profiles. By collecting basic biometric and preference data such as age, weight, fitness goals, and workout intensity, the system leverages the Google Gemini AI model to create structured and customized workout routines along with supporting nutrition tips.

The application is built using a modern full-stack architecture with FastAPI as the backend framework, Jinja2 for templating, and SQLite with SQLAlchemy ORM for data persistence. To ensure safety and reliability, AI-generated content is processed through Markdown rendering and sanitized using Bleach to prevent malicious script injection. The system also implements input validation, structured logging, and centralized error handling to support production-ready software practices.
A key feature of FitBuddy is its feedback-driven improvement loop, allowing users to submit feedback on generated plans so the AI can revise and refine recommendations accordingly. All generated plans and updates are stored in a database, enabling an administrative dashboard for monitoring user records and system activity.

Overall, FitBuddy demonstrates how modern AI technologies can be effectively integrated into a scalable web application to provide personalized fitness guidance. The project highlights the practical use of large language models in health-related applications while maintaining security, usability, and maintainable backend architecture.

2. Problem Statement:

Many individuals want to follow a structured fitness routine but struggle to find workout plans that match their personal goals, physical condition, and preferred training intensity. Most existing fitness applications provide **generic, one-size-fits-all workout programs** that do not adapt to individual user characteristics such as age, weight, and fitness objectives. As a result, users often experience ineffective training, reduced motivation, or difficulty maintaining consistent progress.

Additionally, creating personalized workout plans manually can be time-consuming and requires a certain level of fitness knowledge. Traditional systems also lack mechanisms for **continuous improvement based on user feedback**, making it difficult to adjust routines when users encounter challenges or changing fitness needs.

Therefore, there is a need for a system that can **automatically generate personalized workout plans**, adapt recommendations based on user input and feedback, and present the information in a clear and accessible format. The proposed system, FitBuddy, addresses this problem by using **AI-powered plan generation** to provide customized, flexible, and user-centered fitness guidance.

3. Objectives:

- **Develop an AI-powered web application** that generates personalized 7-day workout plans based on user fitness details and goals.

- **Collect and process user profile information** such as name, age, weight, fitness goal, and workout intensity through a user-friendly interface.

- **Generate customized workout routines and nutrition tips** using an AI model to provide relevant and structured fitness guidance.

- **Store user data and generated plans** in a relational database for record keeping and administrative review.

- **Enable feedback-based plan improvement**, allowing users to submit feedback so the system can update and refine workout plans.

- **Provide secure and reliable system functionality** through proper validation, error handling, and safe rendering of AI-generated content.

4. Scope of the project:

The scope of the FitBuddy project focuses on developing an AI-powered web application that generates personalized 7-day workout plans based on user profile information and fitness goals. The system is designed to demonstrate how artificial intelligence can be integrated with modern web technologies to provide customized fitness guidance.

In Scope:

- Generation of personalized **7-day workout plans** using AI based on user inputs such as age, weight, fitness goal, and workout intensity.
- Generation of **basic nutrition tips** to complement the workout plan.
- **User feedback system** that allows modification or improvement of the generated workout plan.
- **Database storage** of user information and generated workout plans for tracking and review.
- **Admin dashboard** to view stored user records and generated plans.

- **REST API endpoints** for workout plan generation and feedback submission.
- Implementation of **input validation, error handling, and secure rendering** of AI-generated content.

Out of Scope:

- Medical diagnosis or treatment recommendations.
- Integration with **wearable fitness devices** or real-time health monitoring systems.
- **User authentication and authorization systems** such as login and account management.
- Development of **native mobile applications** for Android or iOS.

5. Technology Stack:

Component	Technology Used	Description
Backend Framework	FastAPI (Python)	High-performance web framework used to build the API and server-side logic.
AI Engine	Google Gemini (GenAI SDK)	Generates personalized 7-day workout plans and nutrition tips.
Database	SQLite	Lightweight relational database used to store user data and generated plans.
ORM	SQLAlchemy	Handles database operations and mapping between Python objects and database tables.
Frontend	HTML, CSS, JavaScript	Used to create the user interface and handle client-side interactions.
Template Engine	Jinja2	Dynamically renders HTML pages using backend data.
Data Validation	Pydantic	Validates user input and ensures correct data formats.
Security & Sanitization	Bleach	Cleans AI-generated Markdown content to prevent malicious scripts.
Testing Tools	Pytest, HTTPX	Used for testing API endpoints and application functionality.
Server / Deployment	Uvicorn	ASGI server used to run the FastAPI application.
Environment Management	Python-dotenv	Loads environment variables such as API keys from .env files.

6. System Architecture:

The **FitBuddy system architecture** follows a **layered architecture model**, which separates the application into different functional layers. This design improves maintainability, scalability, and organization by ensuring that each layer performs a specific responsibility.

The architecture consists of four main layers: Presentation Layer, API Layer, Service Layer, and Data Layer.

1. Presentation Layer

The Presentation Layer is responsible for handling **user interaction and displaying information**. It includes the web interface where users enter their fitness details and view the generated workout plans.

Technologies Used:

- HTML
- CSS
- JavaScript
- Jinja2 Templates

This layer collects user inputs such as **name, age, weight, fitness goal, and workout intensity** and sends the request to the backend for processing.

2. API Layer

The API Layer manages **communication between the frontend and backend services**. It processes incoming requests, validates the data, and forwards it to the appropriate service modules.

Responsibilities:

- Handle HTTP requests and responses
- Validate user input using schemas
- Provide REST API endpoints for plan generation and feedback

Technology Used:

- FastAPI

3. Service Layer

The Service Layer contains the **core business logic of the application**. It interacts with the AI model and processes the generated results before sending them to the presentation layer.

Responsibilities:

- Generate personalized workout plans using AI
- Produce nutrition tips
- Process user feedback to update workout plans
- Convert AI-generated Markdown to safe HTML

Technologies Used:

- Google Gemini AI
- Markdown processing
- Bleach for sanitization

4. Data Layer

The Data Layer handles **data storage and database interactions**. It manages user records, generated workout plans, and feedback updates.

Responsibilities:

- Store user details and generated plans
- Retrieve stored records for the admin dashboard
- Maintain database transactions and integrity

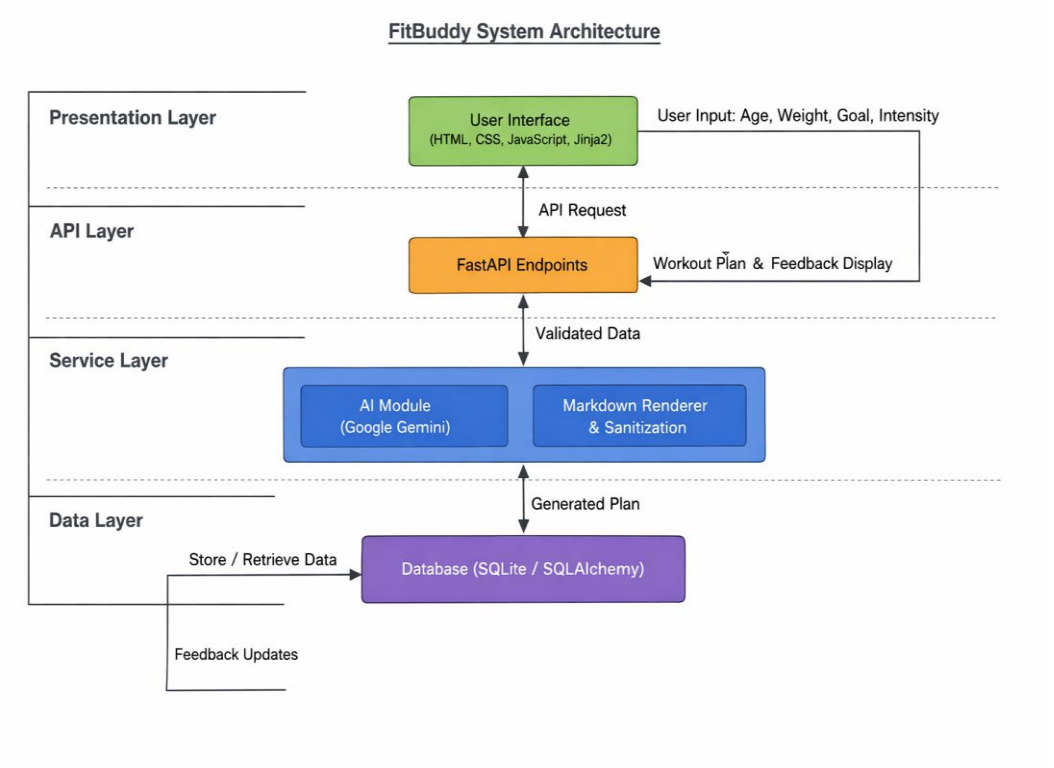
Technologies Used:

- SQLite Database
- SQLAlchemy ORM

Data Flow Overview

1. The user enters fitness information through the web interface.
2. The request is sent to the FastAPI backend through API endpoints.
3. The service layer sends the data to the AI model to generate a workout plan.
4. The generated plan is processed, sanitized, and stored in the database.

5. The final workout plan and nutrition tips are displayed to the user.
6. If feedback is submitted, the system regenerates and updates the plan.



7. Module Description:

The FitBuddy system is organized into multiple modules, each responsible for a specific part of the application. This modular design improves maintainability, scalability, and clarity of the overall system.

1. App Initialization Module (app/main.py)

This module is responsible for **initializing the FastAPI application** and configuring the core system settings.

Functions:

- Creates and configures the FastAPI application.
- Initializes the database connection during application startup.
- Registers middleware such as CORS and security headers.
- Mounts static files and templates.
- Handles global exception configuration.

2. API Routes Module (app/api/routes.py)
This module defines the **main API endpoints** used by the application. It manages communication between the frontend interface and backend services.
Functions:

- Displays the main user input form.
- Processes workout plan generation requests.
- Accepts feedback and updates workout plans.
- Provides admin dashboard data.
- Supports REST API endpoints for external requests.

3. Dependency Management Module (app/api/deps.py)
This module provides **dependency injection utilities** used throughout the application.
Functions:

- Supplies database session instances.
- Provides template rendering utilities.
- Creates and manages AI service instances.
- Ensures required configuration values (such as API keys) are available.

4. AI Service Module (app/services/ai_service.py)
This module contains the **core business logic related to AI-powered plan generation**.
Functions:

- Generates personalized 7-day workout plans using the AI model.
- Produces nutrition tips based on user goals and workout intensity.
- Updates workout plans based on user feedback.
- Handles AI errors such as timeouts or quota limits.

5. Markdown Rendering Module (app/services/markdown_renderer.py)
This module processes the AI-generated Markdown content and converts it into **safe HTML for display in the user interface**.
Functions:

- Converts Markdown text into HTML format.
- Sanitizes output using Bleach to remove unsafe scripts.
- Ensures secure rendering of AI-generated content.

6. Database Module (app/core/database.py)
This module manages **database configuration and connection handling**.
Functions:

- Creates the SQLAlchemy database engine.
- Provides session management for database transactions.
- Initializes database tables at startup.

7. Data Model Module (app/models/user_record.py)

This module defines the **database schema used for storing user data and workout plans**.

Functions:

- Defines the structure of the users table.
- Stores user profile details.
- Stores original and updated workout plans.
- Maintains generation timestamps.

8. Configuration and Security Module (app/core/config.py, app/core/security.py)

This module manages **application settings and security policies**.

Functions:

- Loads environment variables from configuration files.
- Manages API keys and application settings.
- Adds security headers such as Content Security Policy.
- Protects against common web vulnerabilities.

9. Logging and Exception Module (app/core/exceptions.py, app/core/logging_config.py)

This module ensures **proper error handling and system monitoring**.

Functions:

- Defines custom exception classes for AI service errors.
- Handles validation and runtime exceptions.
- Provides structured logging for debugging and monitoring.

8. Database Design

The FitBuddy system uses a relational database (SQLite) to store user information and generated workout plans. The database is managed using SQLAlchemy ORM, which simplifies database operations and object-relational mapping.

The system uses a single primary table named users to maintain user details and AI-generated workout plans.

Table: users

Field Name	*Data Type*	*Description*
id	*Integer (Primary Key)*	*Unique identifier for each user*
name	*String*	*Name of the user*
age	*Integer*	*User's age*
weight	*Float*	*User's body weight*

Field Name	*Data Type*	*Description*
goal	*String*	*Fitness goal (e.g., weight loss, muscle gain)*
intensity	*String*	*Preferred workout intensity*
original_plan	*Text*	*AI-generated workout plan*
updated_plan	*Text*	*Revised plan after feedback*
generated_at	*DateTime*	*Timestamp of plan generation*

This design allows the system to store, retrieve, and update workout plans efficiently while maintaining a history of user interactions.

9. User Interface Description

The FitBuddy application provides a simple and user-friendly web interface that allows users to interact with the system easily.

Main Interface Components

Home Page :

- Displays a form where users enter their personal details such as name, age, weight, fitness goal, and workout intensity.

Result Page

- Displays the AI-generated 7-day workout plan and nutrition tips.
- Includes a feedback section for improving the generated plan.

Admin Dashboard :

- Shows a list of stored user records and generated workout plans.
- Allows administrators to monitor system usage and stored data.

The interface is developed using HTML, CSS, JavaScript, and Jinja2 templates to ensure responsive and dynamic content rendering.

10. API Specification :

FitBuddy provides several REST API endpoints to handle workout plan generation and system interaction.

API Endpoints

POST /api/v1/plans

Purpose: Generate a personalized workout plan.

Input:

- name
- age
- weight
- goal
- intensity

Output:

- user_id
- generated workout plan
- nutrition tip
- generation timestamp

POST /api/v1/plans/feedback

Purpose: Update an existing workout plan using user feedback.

Input:

- user_id
- feedback message

Output:

- updated workout plan
- updated nutrition tip

GET /api/v1/users

Purpose: Retrieve a list of stored user records.

GET /health

Purpose: Check system health status.

Output:

{ "status": "ok" }

11. Validation and Error Handling:

FitBuddy implements strong input validation and error management to ensure system reliability.

Validation

- User input is validated using Pydantic schemas.
- Ensures correct data types and required fields.
- Prevents invalid or incomplete data submissions.

Error Handling

- AI service errors (timeouts or quota issues) are handled gracefully.
- Database errors trigger automatic transaction rollback.
- API responses return clear and structured error messages.

This approach improves system stability and provides better user experience.

12. Security Considerations

Security is an important aspect of the FitBuddy system to protect user data and prevent malicious activity.

Implemented Security Measures

Input Validation

- Prevents malformed or harmful data from entering the system.

Content Sanitization

- AI-generated Markdown is cleaned using Bleach to remove unsafe HTML scripts.

Security Headers

- Content Security Policy (CSP)
- X-Frame-Options
- X-Content-Type-Options

These measures protect the application from common web vulnerabilities such as Cross-Site Scripting (XSS).

13. Testing Strategy

FitBuddy includes a testing setup to ensure the reliability of system functionality.

Testing Tools

- *Pytest – Used for writing and executing test cases.*
- *HTTPX – Used for testing API endpoints.*

Testing Areas

- *API endpoint functionality*
- *Input validation checks*
- *Error handling behavior*
- *Health check endpoint verification*

Future improvements include mock AI testing and database transaction testing.

14. Key Features Implemented

The FitBuddy system includes several important features:

- AI-powered personalized workout plan generation
- Nutrition tip recommendations
- Feedback-based plan improvement

- Secure Markdown rendering and sanitization
- User data storage with SQL database
- Admin dashboard for viewing records
- RESTful API support
- System health monitoring endpoint

These features provide a complete AI-assisted fitness planning platform.

15. Limitations

Despite its functionality, the system has certain limitations.

- Dependence on external AI API availability and quota limits
- No user authentication or login system
- Limited long-term fitness progress tracking
- Workout plans are text-based only
- The database design is optimized for small to medium-scale usage

These limitations can be addressed in future versions of the project.

16. Future Enhancements

Several improvements can be made to enhance the FitBuddy system.

- Implement user authentication and account management
- Add fitness progress tracking with charts
- Integrate wearable devices and health trackers
- Include exercise demonstration videos
- Implement background processing and caching
- Provide multilingual support
- Develop a mobile application version

These enhancements would make the system more scalable and user-friendly.

17. Conclusion

FitBuddy successfully demonstrates the integration of artificial intelligence with modern web technologies to provide personalized fitness guidance.

By combining FastAPI, Google Gemini AI, SQLAlchemy, and secure web practices, the system generates customized workout plans that adapt to user goals and preferences. The feedback mechanism further improves the accuracy and usefulness of generated plans.

The project highlights how AI-powered systems can be applied to health and wellness applications, providing users with practical and personalized fitness assistance.

18. References

1. **FastAPI Documentation**
   https://fastapi.tiangolo.com/
2. **SQLAlchemy Documentation**
   https://docs.sqlalchemy.org/
3. **Pydantic Documentation**
   https://docs.pydantic.dev/
4. **Google GenAI SDK**
   https://ai.google.dev/
5. **Jinja2 Documentation**
   https://jinja.palletsprojects.com/
6. **Bleach Documentation**
   https://bleach.readthedocs.io/
7. **Python Markdown Documentation**
   https://python-markdown.github.io/

19. Appendix

A. Environment Variables

- GOOGLE_API_KEY
- GEMINI_MODEL_CANDIDATES
- CORS_ALLOW_ORIGINS

B. Project Structure

```
app/
templates/
```

static/
requirements.txt

C. Sample Run Commands

```
pip install -r requirements.txt
uvicorn app.main:app --reload
```